

A menos de aclaración en contrario, deben especificarse y programarse las macros que se usen en programas χ o Imp o en máquinas de Turing.

Problema 1. El cálculo λ puro y la correspondiente operación de *sustitución* se pueden representar en Haskell de la siguiente manera:

```
data Lambda = X Nat
            | A Lambda Lambda
            | L Nat Lambda

sust :: Nat -> Lambda -> Lambda -> Lambda
sust y n v@(X x)
    | y == x = n
    | otherwise = v
sust y n (A m1 m2) = A (sust y n m1) (sust y n m2)
sust y n f@(L x m)
    | y == x = f
    | otherwise = L x (sust y n m)
```

Notas:

- `sust y n m` sustituye en el término `m` las ocurrencias libres de la variable de nombre `y` por el término (asumido cerrado) `n`.
- El tipo `Nat` provee una representación cualquiera de números naturales.

Se pide:

1. Traducir el código precedente a χ .
2. Ídem a Imp .

En cada caso se podrá hacer uso de una macro para la igualdad de naturales. Estas macros deberán ser especificadas (es decir, se deberá aclarar su forma de uso).

Problema 2.

1. Haciendo uso de una reducción adecuada (de alguna versión del *halting problem*) demostrar que es indecidible en χ si un programa dado no computa **False** para ninguna entrada.
2. Demostrar, haciendo uso del *Teorema de Rice*, que es indecidible si un programa dado computa únicamente números naturales pares o bien únicamente números naturales impares.

Problema 3. Se consideran expresiones binarias (es decir, árboles conformados por variables, literales (0's y 1's) y las operaciones opuesto y producto). Estas expresiones se evalúan sobre una memoria representada como una tabla en donde a cada variable se le asigna un bit.

1. Demostrar que el problema *E0* que decide si una expresión dada tiene valor 0 para algún estado de la memoria es *NP*.
2. Indique qué procedimiento seguiría para probar que este problema es *NP-Completo*.